



Hello Java!

Let's Code Together



by Aditya Varma

Preface

Index

- 📖 Introduction
- 📖 Java Basics
- 📖 Data Types & Variables
- 📖 Operators in Java
- 📖 Control Flow
- 📖 Conditionals in Java
- 📖 Loops in Java
- 📖 Arrays in Java
- 📖 Strings in Java
- 📖 Methods in Java
- 📖 Introduction to OOP in Java
- 📖 Packages in Java
- 📖 Exception Handling in Java
- 📖 File Handling in Java
- 📖 Projects

Introduction

//Introduction to Java

What is Java?

Java is a high-level, general-purpose programming language used to build everything from small applications to massive enterprise systems. You can think of Java as a universal translator, you write code once, and it automatically adapts to different devices and operating systems.

Java belongs to the category of compiled languages.

This means:

- You write code in Java.
- A compiler converts it into a special format called bytecode.
- This bytecode is then executed by the Java Virtual Machine (JVM) on any device.

You can imagine Java's compiler as a chef who prepares a basic dish (bytecode), and the JVM is like local cooks in each country adding their own flair depending on their kitchen (OS/platform).

The recipe stays the same, only the cooking varies.

Why is bytecode important?

Because bytecode is not tied to any specific device. It's a "universal" format that works everywhere, that's where Java gets its magic.

Features of Java!

1. Platform Independence

Java's famous idea is "Write Once, Run Anywhere" (WORA).

This means:

- You write Java code once.
- It gets converted into bytecode.
- That bytecode runs on any system where a JVM exists — Windows, Linux, macOS, Android, etc.

Think of bytecode like a movie subtitle file, the same subtitle file works on many different media players. Each player interprets it in its own way, but you don't need to create separate subtitles for each one. This reduces work for programmers, no need to maintain different code versions for each platform.

2. OOP Language (Object-Oriented Programming)

Java uses the object-oriented approach, which means it organizes programs into “objects”, mini units that combine data and functions.

You can imagine objects like real-world things:

- A Car object has data (color, model) and actions (start, brake).
- A Student object has data (name, age) and actions (study, takeExam).

OOP helps keep programs clean, organized, and easier to expand.

3. Secure & Robust

Java takes security seriously.

It avoids many common programming hazards because:

- Memory management is automatic
- Many risky operations are restricted
- Programs run inside the JVM “sandbox,” giving an extra layer of protection

Java also does automatic memory handling:

- When you create an object, Java automatically allocates memory for it.
- When the object is no longer used, the Garbage Collector recycles that memory.

You can think of it like a house where a cleaning robot (GC) quietly removes stuff you no longer need, preventing a messy room (memory leak).

This makes Java stable (robust) and safe.

4. Portable

Java programs can move from one computer to another without modification.

Since the bytecode stays the same everywhere, Java apps don’t care whether they’re running on:

- A laptop
- A server
- A mobile device
- A smart TV

It’s as portable as carrying a USB drive with universal files plug in anywhere, and it works.

What Java is Used For?

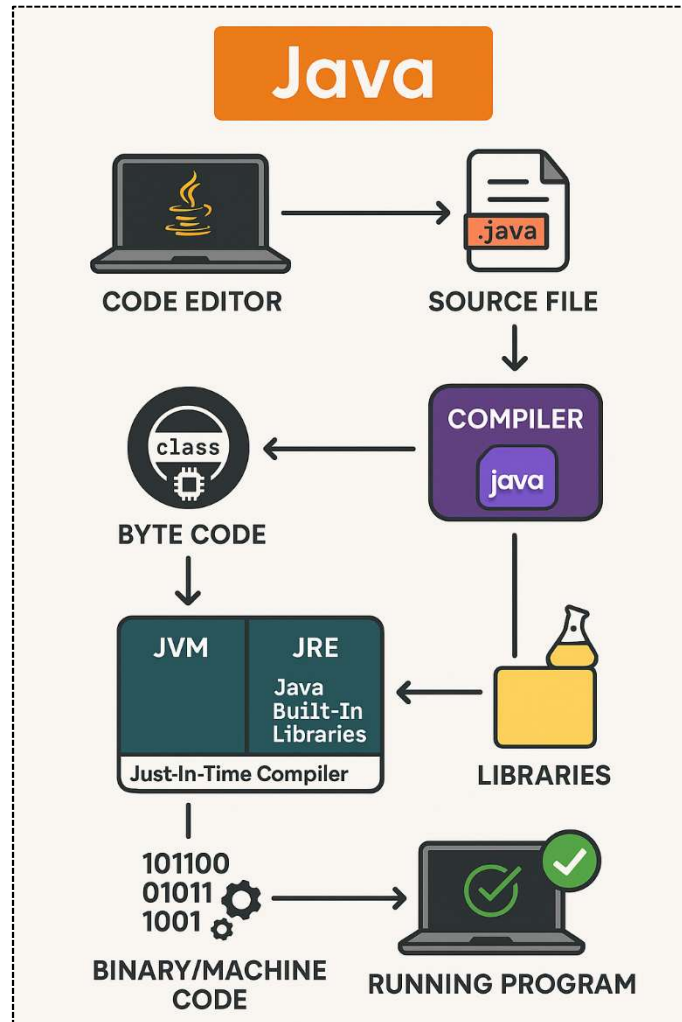
Java has been around for decades and still powers a huge part of the software world. It’s used in:

-  Enterprise software (banking systems, ERPs, corporate tools)
-  Android apps (Java was the primary Android language for years)
-  Web applications (back-end servers using Java)
-  Cloud-based systems
-  Games (lightweight games and certain engines)
-  Desktop applications

- 🖱️ Embedded systems (smart cards, sensors, appliances)

Basically, Java is like a Swiss Army Knife — you'll find it almost everywhere in the tech world.

Write Once, Run Anywhere



//JVM, JRE & JDK – The Three Pillars of Java

When learning Java, these three names show up everywhere. They sound technical, but once you see them as parts of a mini Java ecosystem, everything becomes easy to understand.

Let's break them down with a simple analogy:

1. JVM – Java Virtual Machine

Think of the JVM as a translator who understands bytecode.

- You give JVM the bytecode produced by the compiler.
- JVM reads it, understands it, and turns it into instructions your computer can execute.
- Each operating system has its own version of the JVM, but they all understand the same bytecode.

Simple Analogy:

Imagine you write a letter in a universal language. The JVM is like a local interpreter who reads the letter and speaks in the local language (Windows, Linux, Mac). That's how Java achieves Write Once, Run Anywhere.

2. JRE – Java Runtime Environment

The JRE is the “playground” where Java programs actually run.

It includes:

- The JVM
- Built-in Java libraries (pre-written code you can use)
- Tools needed to run programs

But note: You cannot write or compile Java code with the JRE. It's only for running applications.

Analogy:

If JVM is the performer, then JRE is the stage + lights + tools that allow the performer to do their job.

3. JDK – Java Development Kit

The JDK is the complete toolset for people who want to write Java programs.

It contains:

- Everything inside the JRE
- Plus, tools for developers
 - Java compiler (javac)
 - Debuggers
 - Documentation tools

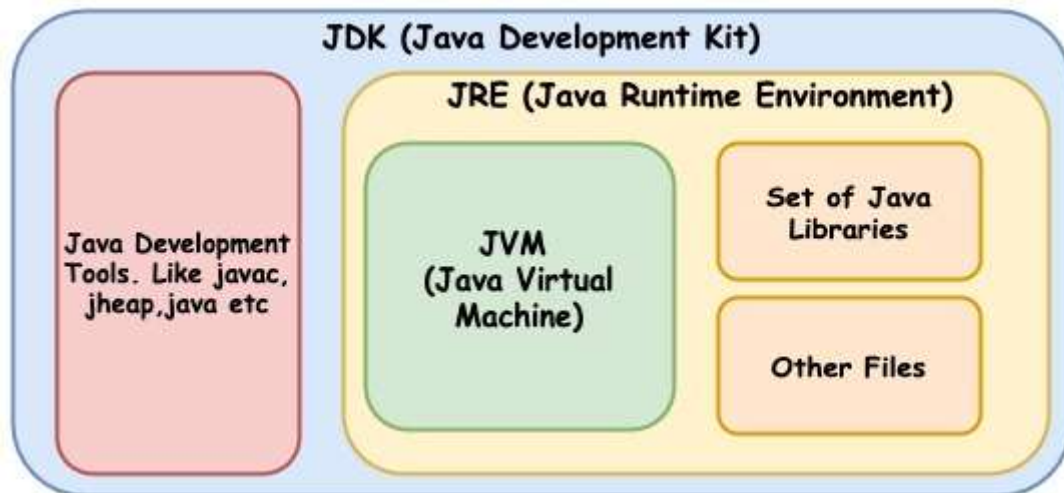
Analogy:

If JRE is the stage, then JDK is the entire theatre building, including:

- Stage (JRE)
- Backstage tools

- Equipment
- Workshops

JDK = Everything you need to create, compile, and run Java code.



//Installing Java (JDK)

Check if Java is already installed:

On Windows:

1. Open Command Prompt (CMD)
2. Type:

```
java -version
```

If Java is installed, you will see something like:

```
java version "17.0.10"
```

If Java is NOT installed, you will see:

```
'java' is not recognized as an internal or external command
```

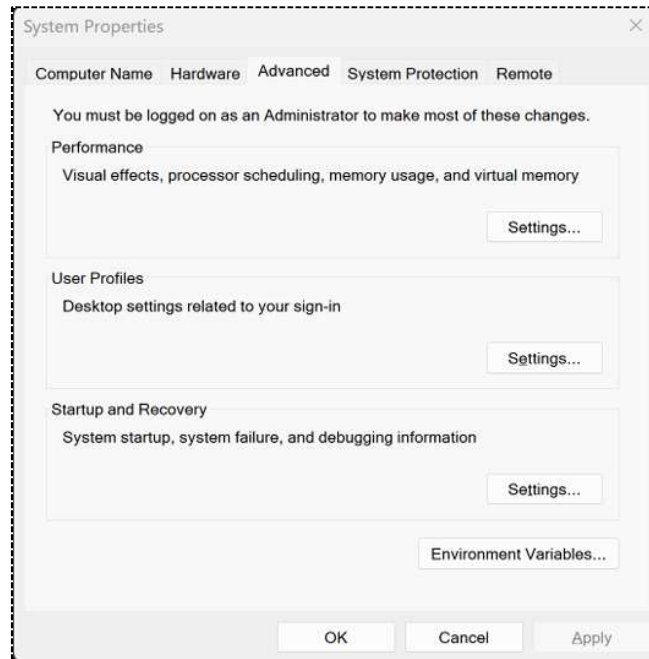
Installing Java (JDK):

1. Visit the Oracle Java SE Downloads page at <https://www.oracle.com/java/technologies/downloads/>.
2. Select the appropriate installer for your Windows operating system (for example, Windows x64 Installer).
3. Download the installer by clicking on the file link.
4. Run the downloaded installer (the .exe file) and follow the on-screen instructions to complete the installation.

Setting Up Environment Variables:

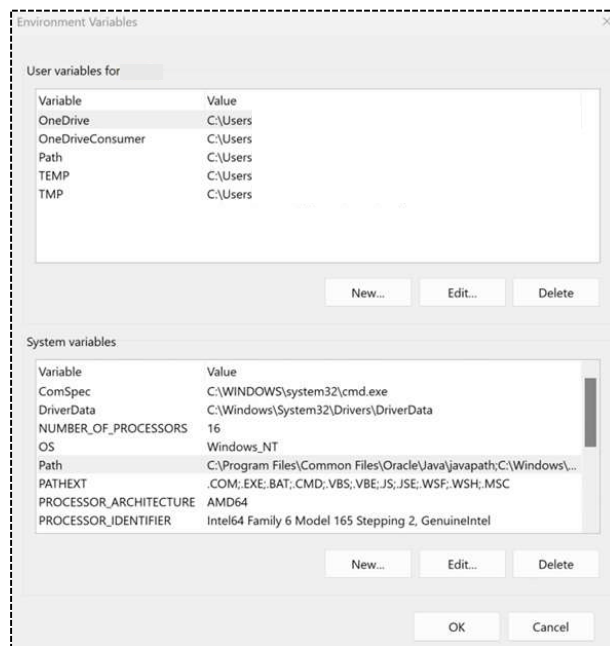
(This step is sometimes done automatically by newer installers, but you should know it.)

1. Find the JDK installation path:
Example path:
C:\Program Files\Java\jdk-25
2. Add JAVA_HOME:
 - Search “Edit the system environment variables” -> Click Environment Variables
 - Under System variables, click New
 - Variable name: JAVA_HOME
 - Variable value: (your JDK folder path): C:\Program Files\Java\jdk-25



3. Add JDK to PATH:

- Go to System variables → Path → Edit
- Click New, add: %JAVA_HOME%\bin



4. Verify installation:

- Open CMD and type:

```
java -version
```

You should see version numbers.

//Installing an IDE (VS Code / IntelliJ)

Option A: VS Code

1. Download and install VS Code
2. Open VS Code → Go to Extensions
3. Install:
 - Extension Pack for Java (important)
 - Debugger for Java
 - Java Test Runner (optional)

That's it VS Code is ready for Java development.

Option B: IntelliJ IDEA

(More beginner-friendly than VS Code. Handles everything automatically.)

1. Download IntelliJ IDEA Community Edition (free)
2. Install and open it
3. IntelliJ automatically detects your JDK
 - If it asks: just select the JDK folder (example: C:\Program Files\Java\jdk-17)
4. Create a New Java Project and start coding.

//Creating & Running Your First Java Program

Using CMD (Command Prompt):

Step 1: Create a new file named Hello.java in notepad.

Java requires the filename and class name to be the same.

Step 2: Write this code in the notepad,

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Step 3: Save the file in a folder you want.

Step 4: Open CMD and go to that folder.

```
cd C:\JavaPrograms
```

Step 5: Compile the file

```
javac Hello.java
```

This will create:

Hello.class (Your bytecode file)

Step 6: Run the program

```
java Hello
```

Output:

```
Hello World
```

Using IntelliJ IDEA (recommended):

Step 1: Open IntelliJ → Create New Project

- Select New Project
- Choose Java
- Select JDK 17
- Click Create

Step 2: Create a Java File

Inside src folder:

1. Right-click src
2. Select New → Java Class
3. Name it:

Step 3: Write the Program

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Step 4: Run the Program

At the top-left of the editor you will see a green play button ►. You will get the Output.

//Basic Structure of a Java Program

Every Java program follows a specific structure. Once you understand this structure, Java becomes much easier to read and write.

Let's break it down part by part.

Understanding Class Structure:

Every Java file contains **at least one class**.

General Structure:

```
public class ClassName {  
    // variables (optional)  
    // methods (optional)  
}
```

Points to remember:

- The class name must match the file name
e.g., file: `Hello.java` → class: `Hello`
- A class is like a container that holds your code.
- Everything in Java lives inside a class.

Example: Think of a class as a **house**, and all your code sits inside it.

The `main()` Method Explained:

This is the starting point of **every** Java program.

```
public static void main(String[] args) {  
    // program starts here  
}
```

What each word means:

- `public` → Anyone can run this method
- `static` → Runs without creating an object
- `void` → Does not return anything
- `main` → Special method where program starts
- `String[] args` → Accepts inputs from the user (command-line arguments)

Think of `main()` as:

The **front door** of your program. Java starts running your code by entering through this door.

Compiling & Running a Java Program:

Compilation: Java code (Hello.java) → compiled into bytecode → Hello.class

```
javac Hello.java
```

Execution: Run the bytecode using the JVM

```
java Hello
```

Common Errors Beginners Face:

1. File name does not match class name:

```
public class Hello {}
```

but file is named: HelloWorld.java

Fix: Rename file to Hello.java

2. Missing semicolon

3. Wrong capitalization:

Java is **case-sensitive**.

```
System ≠ system
```

```
main ≠ Main
```

```
String ≠ string
```

Fix: Use exact uppercase/lowercase.

4. Missing curly braces { }:

Every opening brace { must have a closing brace }.

5. Running the program with .java extension:

You never include the .java when running.

6. Typing code outside the class:

Everything must be **inside the class** and mostly inside the **main method** for beginners.

Quick Visual Summary:

```
public class Hello {           ← Class begins
    public static void main(String[] args) { ← Main method begins
        System.out.println("Hello World"); ← Statement
    }                               ← Main closes
}                                   ← Class closes
```

//Understanding Bytecode & Java Compilation Process

When you write a Java program, you type code in English-like syntax inside a .java file. But your computer cannot understand Java directly. Computers understand machine code (0s and 1s). So, Java uses a two-step process that makes it special:

1. Java Compilation: From .java → .class

Whenever you compile a Java file using:

```
javac Program.java
```

the Java compiler (JAVAC) converts your source code into bytecode, stored in a .class file.

What is Bytecode?

- Bytecode is a universal, platform-independent code.
- It is not machine code, and not Java code — it is something in between.
- Every Java program is first converted to bytecode.

Why does Java use bytecode?

Because bytecode can run on any operating system — Windows, macOS, Linux — as long as the system has a JVM. This is why Java is called:

“Write Once, Run Anywhere (WORA)”

2. JVM Execution: From Bytecode → Machine Code

Once bytecode is created, the Java Virtual Machine (JVM) takes over.

When you run:

```
java Program
```

JVM loads the .class file and performs two main tasks:

1. Bytecode Verification

Checks if the bytecode is:

- safe
- valid
- not trying to access memory illegally
- secure for execution

2. Bytecode Execution

JVM converts bytecode into machine code specific to your operating system and CPU.

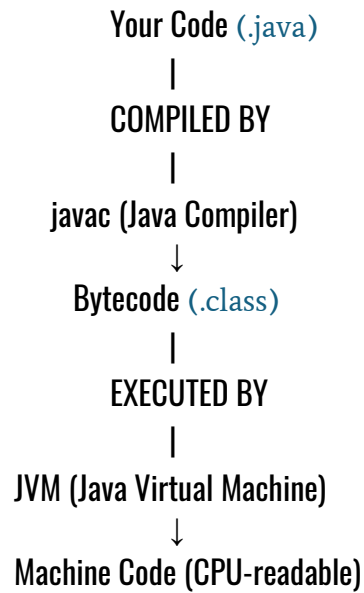
Different systems have different JVMs:

- Windows JVM
- Linux JVM
- macOS JVM

But all understand the same bytecode, which is the magic of platform independence.

How the Full Process Looks:

Here is the entire journey of a Java program simplified:



Why This Process Is Amazing:

- ✓ Portable — runs on any device
- ✓ Secure — JVM checks code before running
- ✓ Optimized — JVM improves performance at runtime
- ✓ Reliable — same program behaves the same everywhere

Java Basics

// Java Tokens

When you read a sentence, your brain breaks it into small parts—words, symbols, punctuation. Java also does the same. A Java program is built using tiny building blocks called *tokens*. These are the smallest units that still have meaning in code.

Think of tokens like LEGO pieces—small on their own, but powerful when connected. Java has 4 major types of tokens:

1. Keywords
2. Identifiers
3. Literals
4. Symbols & Operators

Let's explore them one by one.

Keywords:

Keywords are special words that Java has reserved for its own use. You cannot use them as variable names or class names because Java already knows what these words mean.

Examples:

`class, public, static, int, void, if, else, return`

Analogy:

Think of keywords as “VIP seats” in a theatre—reserved and untouchable.

Identifiers:

Identifiers are the names you give to things in your program:

- Class names
- Variable names
- Method names
- Object names

Rules (easy to remember):

- Can use letters, digits, `_`, `$`
- Cannot start with a digit
- Cannot use spaces
- Cannot use keywords
- Case-sensitive (`Name`, `name`, and `NAME` are different)

Identifiers are like labels you stick on your storage boxes so you know what's inside.

Literals:

Literals are fixed values written directly in the code. They don't change.

Types of literals:

- `10` → integer literal
- `3.14` → floating literal
- `'A'` → character literal
- `"Hello"` → string literal
- `true`, `false` → boolean literal

These are the raw values that your program works with.

Symbols & Operators:

These are characters that help Java understand code structure and perform actions.

Common symbols:

- `()` → parentheses
- `{}` → braces
- `[]` → brackets
- `;` → statement end
- `" "` → string quotes

Operators do the work:

- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Relational: `==`, `!=`, `>`, `<`
- Logical: `&&`, `||`, `!`
- Assignment: `=`, `+=`, `-=`

Think of operators as the tools your program uses to get things done.

Why Tokens Matter?

Tokens are the foundation of Java syntax. If Java were a language, tokens would be its alphabet. Understanding them helps you read, write, and debug programs correctly.

Lesson Program: Tokens in Java.

Source Code:

Tokens.java

```
1 // Program: Tokens in Java
2
3 public class Tokens {
4     public static void main(String[] args) {
5
6         System.out.println("\n=== TOKENS IN JAVA ===");
7
8         System.out.println("""
9 Java programs are made of tokens:
10 1. Keywords
11 2. Identifiers
12 3. Literals
13 4. Symbols & Operators
14 """);
15
16         System.out.println("Keyword Example: public, class, static");
17         System.out.println("Identifier Example: TokensInJava, count, value");
18
19         System.out.println("Literals:");
20         System.out.println(10);
21         System.out.println(3.14);
22         System.out.println('A');
23         System.out.println("Hello Java");
24         System.out.println(true);
25
26         System.out.println("Symbols & Operators:");
27         System.out.println("Semicolon ; ends statements");
28         System.out.println("Using + operator: " + ("Java" + " Tokens"));
29     }
30 }
```

//Comments in Java

When programmers write code, they also leave notes to themselves or others—just like writing sticky notes on a book page.

In Java, these notes are called comments, and the important part is:

The computer completely ignores them. They exist only for humans.

Why do we use comments?

- To explain the logic
- To make code readable
- To give instructions or warnings
- To turn OFF a line temporarily without deleting it

Java gives us 3 types of comments:

Single-Line Comment (//)

A single-line comment starts with `//`. Everything after it on the same line gets ignored.

Used for:

- Quick notes
- Explaining one line
- Temporarily disabling a line

Think of it as whispering a small reminder in your code.

Multi-Line Comment (/* ... */)

Starts with `/*` and ends with `*/`. Anything between them is ignored.

Used for:

- Long explanations
- Notes across multiple lines
- Disabling a block of code

Think of it as wrapping text inside a “big comment box.”

Documentation Comment (/ ... */)**

This is a special style used for generating official Java documentation using the [javadoc](#) tool.

Inside it, special tags are used:

- `@author`
- `@version`
- `@param`
- `@return`

These are used mostly in professional Java projects. Think of them as comment blocks that turn into real documentation pages.

Lesson Program: Comments in Java.

Source Code:

Comments.java

```
1 // Program: Comments in Java
2
3 public class Comments {
4     public static void main(String[] args) {
5
6         System.out.println("\n=== COMMENTS IN JAVA ===");
7
8         System.out.println("""
9 Types of comments:
10 1. Single-line
11 2. Multi-line
12 3. Documentation comments
13 """);
14
15         // Single-line comment example
16         System.out.println("Single-line comment example printed.");
17
18         /*
19          This is a multi-line comment.
20          Used for long explanations.
21         */
22         System.out.println("Multi-line comment example printed.");
23
24         /**
25          * Documentation comments help create official docs.
26          */
27         System.out.println("Documentation comment example printed.");
28     }
29 }
```

//Naming Conventions in Java

Naming conventions are simply rules for naming things in your Java program. They don't affect how the code runs, but they affect how it looks, how easy it is to understand, and how professional it feels. Java programs contain names for:

- classes
- methods
- variables
- constants
- packages
- interfaces

Because Java is case-sensitive, following naming conventions becomes very important. These conventions ensure your code is: Clean, Predictable, Easy to understand, Standardized across developers

Let's explore each naming rule one by one.

1. Class Naming Convention

Classes represent objects or entities, so their names should be nouns. Use PascalCase (UpperCamelCase), First letter of every word is capital, Name must be meaningful

Examples (correct):

- `StudentRecord`
- `OrderDetails`
- `PaymentGateway`

2. Method Naming Convention

Methods perform actions, so they should be verbs. Use camelCase, first word lowercase, next words start with uppercase

Correct:

- `calculateTotal()`
- `getName()`
- `displayMessage()`

3. Variable Naming Convention

Variables hold values. Their names must be clear and descriptive. Use camelCase, start with a letter, not a digit, avoid very short names (except i, j in loops), Should be easy to understand

Correct:

- `studentAge`
- `totalMarks`
- `numberOfItems`

4. Constant Naming Convention

Constants represent fixed values. Use UPPERCASE, use underscores between words, declare them using `final`

Correct:

- `MAX_SPEED`
- `PI_VALUE`

5. Package Naming Convention

Packages represent folder structures. Use all lowercase, Use meaningful names, Companies use reverse domain naming

Correct:

- `student.management`
- `com.school.app`

6. File Naming Convention

A Java file must match the public class name.

Lesson Program: Naming Conventions in Java.

Source Code:

Comments.java

```
1 // Program: Java Naming Conventions
2
3 public class NamingConvention {
4     public static void main(String[] args) {
5
6         System.out.println("\n=== JAVA NAMING CONVENTIONS ===");
7
8         System.out.println("""
9 1. Class → PascalCase
10 2. Method → camelCase
11 3. Variable → camelCase
12 4. Constant → UPPER_CASE
13 5. Package → lowercase
14 6. Interface → PascalCase
15 """);
16
17         System.out.println("Correct Class Name: StudentData");
18         System.out.println("Correct Method Name: calculateMarks()");
19         System.out.println("Correct Variable Name: totalPrice");
20         System.out.println("Correct Constant Name: MAX_LIMIT");
21         System.out.println("Correct Package Name: student.management");
22     }
23 }
```

//Escape Sequence in Java

When we write text inside `System.out.println()`, we usually print normal characters like letters, digits, and symbols.

But sometimes we need to print:

- a new line
- a tab space
- a double quote inside the string
- a backslash
- specially formatted text

Normal characters cannot do this directly. So Java gives us special character combinations called escape sequences.

What are Escape Sequences?

Escape sequences are special characters written using a backslash (`\`). They allow Java to print characters that are otherwise difficult or impossible to write directly.

Example:

`\"` prints a double quote

`\\` prints a backslash

`\n` prints a new line

Common Escape Sequences in Java

1. `\n` New Line

- Moves the output to the next line.
- Works like pressing the Enter key.

2. `\t` Tab Space

- Inserts a horizontal tab (same as pressing TAB).
- Used to align data neatly.

3. `\"` Double Quote

- Allows printing `"` inside a string.
- Without escape sequence, Java thinks the string ends.

4. `\\` Backslash

- Prints a single backslash.
- Because a single backslash starts an escape sequence, we need double backslash.

Why Escape Sequences Are Useful?

- They help in:
- Pretty formatting
- Writing multi-line messages
- Printing quotes inside text
- Drawing patterns
- Making console output cleaner

Escape sequences make your output professional and readable.

Lesson Program: Escape Sequence in Java.

Source Code:

EscapeSequences.java

```
1 // Program: Escape Sequences in Java
2
3 public class EscapeSequences {
4     public static void main(String[] args) {
5
6         System.out.println("Line1\nLine2");           // \n -> New line
7         System.out.println("Name:\tJohn");             // \t -> Tab
8         System.out.println("He said, \"Hello\"");       // \" -> Double quote
9         System.out.println("Path: C:\\Files");          // \\ -> Backslash
10
11         // Combined example
12         System.out.println("Start\n\tMiddle\n\t\tEnd");
13     }
14 }
```

Lesson Program: Variables and Constants in Java.

Source Code:

VariablesAndConstants.java

```
1 // Program: Variables and Constants in Java
2
3 public class VariablesAndConstants {
4     public static void main(String[] args) {
5
6         // variable declaration + initialization
7         int age = 25;
8         double price = 199.99;
9         String name = "John";
10
11        // using variables
12        System.out.println("Age: " + age);
13        System.out.println("Price: " + price);
14        System.out.println("Name: " + name);
15
16        // constants using final
17        final int MAX_STUDENTS = 100;
18        final double PI = 3.14159;
19
20        System.out.println("Max Students: " + MAX_STUDENTS);
21        System.out.println("PI Value: " + PI);
22
23        // variable changes allowed
24        age = 26;
25        System.out.println("Updated Age: " + age);
26
27        // MAX_STUDENTS = 200; // ✗ Not allowed (constant)
28    }
29 }
```

Lesson Program: Data Types in Java.

Source Code:

DataTypes.java

```
1 // Program: Data Types in Java
2
3 public class DataTypes {
4     public static void main(String[] args) {
5         // primitive data types
6         byte b = 10;
7         short s = 200;
8         int num = 5000;
9         long bigNum = 123456789L;
10        float f = 5.75f;
11        double d = 99.99;
12        char ch = 'J';
13        boolean flag = true;
14
15        System.out.println("byte: " + b);
16        System.out.println("short: " + s);
17        System.out.println("int: " + num);
18        System.out.println("long: " + bigNum);
19        System.out.println("float: " + f);
20        System.out.println("double: " + d);
21        System.out.println("char: " + ch);
22        System.out.println("boolean: " + flag);
23
24        // non-primitive types
25        String name = "John Doe";
26        int[] marks = {90, 85, 88};
27
28        System.out.println("String: " + name);
29        System.out.println("String length: " + name.length());
30        System.out.println("Array marks: " + marks[0] + ", " + marks[1] + ", " + marks[2]);
31    }
32 }
```

Lesson Program: Primitive Data Types in Java.**Source Code:**

PrimitvieDataTypes.java

```
1 // Program: Primitive Data Types
2
3 import java.util.Scanner;
4
5 public class PrimitiveDataTypes {
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9
10        // byte
11        byte b = 100;
12        System.out.println("byte: " + b);
13        System.out.print("Enter byte: ");
14        byte userByte = sc.nextByte();
15        System.out.println("You entered: " + userByte);
16
17        // short
18        short s = 30000;
19        System.out.println("short: " + s);
20        System.out.print("Enter short: ");
21        short userShort = sc.nextShort();
22        System.out.println("You entered: " + userShort);
23
24        // int
25        int i = 95;
26        System.out.println("int: " + i);
27        System.out.print("Enter int: ");
28        int userInt = sc.nextInt();
29        System.out.println("You entered: " + userInt);
30
31        // long
32        long l = 8000000000L;
33        System.out.println("long: " + l);
34        System.out.print("Enter long: ");
35        long userLong = sc.nextLong();
36        System.out.println("You entered: " + userLong);
37    }
```

```
38         // float
39         float f = 99.75f;
40         System.out.println("float: " + f);
41         System.out.print("Enter float: ");
42         float userFloat = sc.nextFloat();
43         System.out.println("You entered: " + userFloat);
44
45         // double
46         double d = 55000.456;
47         System.out.println("double: " + d);
48         System.out.print("Enter double: ");
49         double userDouble = sc.nextDouble();
50         System.out.println("You entered: " + userDouble);
51
52         // char
53         char c1 = 'J';
54         char c2 = 65;
55         System.out.println("char 1: " + c1);
56         System.out.println("char 2: " + c2);
57         System.out.print("Enter char: ");
58         char userChar = sc.next().charAt(0);
59         System.out.println("You entered: " + userChar);
60
61         // boolean
62         boolean flag = true;
63         System.out.println("boolean: " + flag);
64         System.out.print("Enter boolean: ");
65         boolean userBoolean = sc.nextBoolean();
66         System.out.println("You entered: " + userBoolean);
67
68         sc.close();
69     }
70 }
```


Lesson Program: Input Output in Java.**Source Code:**

InputOutput.java

```

1 // Program: Input and Output in Java
2
3 import java.util.Scanner;
4
5 public class InputOutput {
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9
10        // ----- Output Examples -----
11        System.out.println("Welcome to Java I/O Demo!");
12        System.out.print("This is printed on the same line. "); // System.out.print stays on the same line
13        System.out.println("Now we move to the next line."); // System.out.println moves to next line
14        System.out.printf("Formatted number: %.2f\n", 123.456); // printf example for formatted output
15
16        // ----- Input Examples -----
17        // Example 1: Reading a String
18        System.out.print("Enter your name: ");
19        String name = sc.nextLine();
20
21        // Example 2: Reading an integer
22        System.out.print("Enter your age: ");
23        int age = sc.nextInt();
24
25        // Example 3: Reading a double
26        System.out.print("Enter your height in meters: ");
27        double height = sc.nextDouble();
28
29        sc.nextLine(); // clear buffer before reading nextLine()
30
31        // Example 4: Reading a full sentence
32        System.out.print("Enter your city: ");
33        String city = sc.nextLine();
34
35        // Example 5: Reading a boolean
36        System.out.print("Are you a student? (true/false): ");
37        boolean isStudent = sc.nextBoolean();
38
39        sc.nextLine(); // clear buffer before nextLine()
40
41        // Example 6: Reading multiple words (full line)
42        System.out.print("Enter your favorite quote: ");
43        String quote = sc.nextLine();
44
45        // ----- Display Collected Inputs -----
46        System.out.println("\n==== USER DETAILS =====");
47        System.out.println("Name: " + name);
48        System.out.println("Age: " + age);
49        System.out.printf("Height: %.2f meters\n", height);
50        System.out.println("City: " + city);
51        System.out.println("Student: " + isStudent);
52        System.out.println("Favorite Quote: " + quote);
53
54        sc.close();
55    }
56 }

```